Automatisation de requêtes SQL

Présentation, architecture et détails techniques

bash | MySQL 8.4 | Docker Compose | cron

Un script bash générique exécute, à heure fixe, des mises à jour SQL ciblées : il détecte les lignes concernées avec un **SELECT**, puis applique un **UPDATE** sur la liste d'id obtenue et journalise chaque étape. Toute la logique métier vit dans des fichiers `.sql` ; le script ne contient aucune requête.

Élément	Valeur
Base	MySQL 8.4, base *ecommerce* (client, produit, commande, livraison, historique)
Script	`select_update.sh`, bash, aucune dépendance hors client `mysql`
Jobs fournis	commandes, livraisons, produits (un dossier = un job)
Planification	cron hôte, toutes les heures pleines de 6h à 22h
Journalisation	un fichier de log par jour, rétention 7 jours

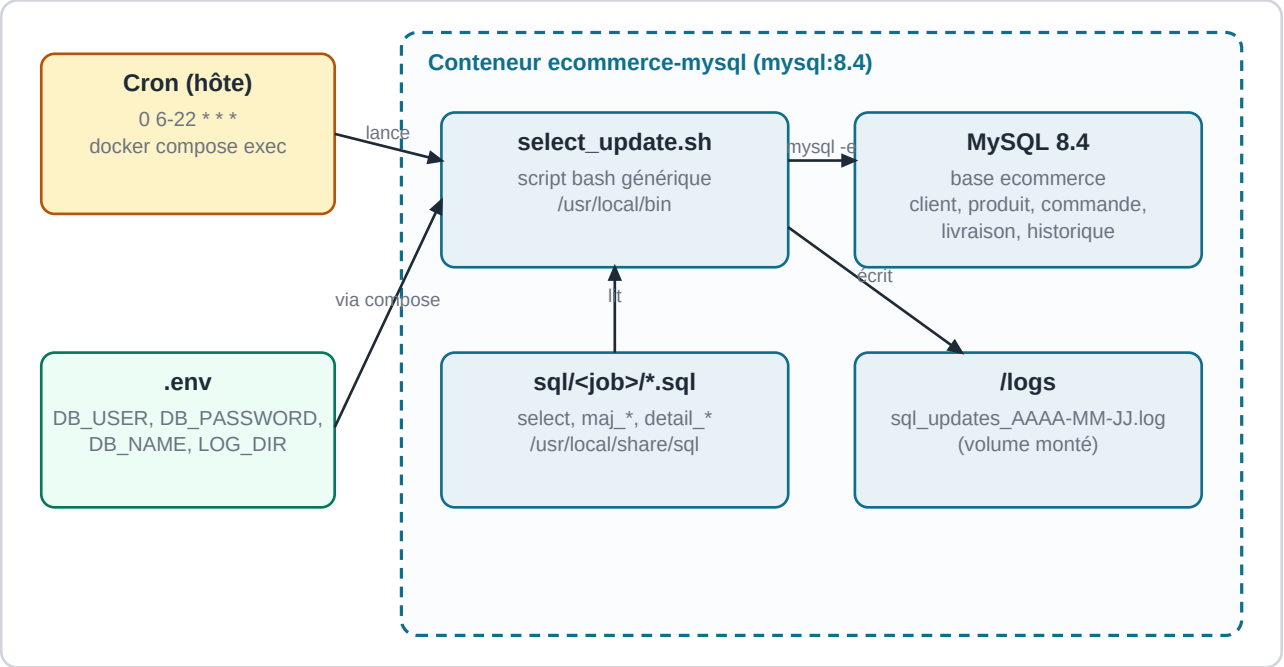
1. Objectif et principe

Automatiser des corrections de données récurrentes sans modifier le code à chaque nouveau besoin. Exemples fournis :

- **commandes** : commandes *ABONDONNE* de clients français passées à *OK*.
- **livraisons** : livraisons *EN_ATTENTE* de clients italiens passées à *EXPEDIE* avec la date du jour.
- **produits** : produits dont le stock est inférieur à 10 réapprovisionnés de 50 unités.

Principe : **détecter, lister, appliquer, tracer**. L'UPDATE ne porte que sur les id détectés par le SELECT, ce qui rend chaque exécution ciblée et auditable.

2. Architecture



Le conteneur embarque le serveur MySQL, le script et les dossiers SQL (copiés dans l'image au build). Le cron reste sur l'hôte et déclenche le script dans le conteneur. Les logs sont écrits dans un volume monté, donc lisibles depuis l'hôte.

Fichier	Role
Dockerfile	Image mysql:8.4, copie de init.sql, init_user.sh, sql/ et du script
docker-compose.yml	Service mysql, variables issues du .env, volume de logs, healthcheck
init.sql	Tables, jeu de données, table historique et triggers (1er démarrage)
init_user.sh	Crée DB_USER avec tous les droits sur DB_NAME s'il n'est pas root
select_update.sh	Script générique : parcourt sql/, exécute les jobs, journalise
sql/<job>/	Requêtes d'un job (détection, mise à jour, détail)
crontab	Planification 0 6-22 * * *

3. Déroulement d'une exécution

Détail avant / après l'UPDATE (si detail_*.sql), le tout journalisé dans le log du jour

- **Attente** : jusqu'à 30 tentatives espacées de 2 s tant que la base n'accepte pas de connexion.
- **Retention** : les fichiers `sql_updates_*.log` plus vieux que `RETENTION_JOURS` sont supprimés et listés dans le log.
- **Sélection** : le SELECT du job est lancé ; sans ligne, le job s'arrête proprement (message dans le log).
- **Liste d'id** : la 1re colonne est extraite (awk + `paste -sd,`) en `4,9,15`.
- **Mise à jour** : le marqueur `{{IDS}}` du fichier maj_*.sql est remplacé par cette liste puis la requête est exécutée.

Convention de dossier

Fichier	Rôle	Obligatoire
`maj_*.sql`	UPDATE, contient {{IDS}}	oui, un seul
`detail_*.sql`	SELECT de contrôle, exécuté avant et après l'UPDATE	non, un seul max
autre `*.sql`	SELECT de détection, 1re colonne = id	oui, un seul

Un dossier qui ne respecte pas cette convention est ignoré avec un message dans le log. Ajouter un job = creer un dossier `sql/<domaine>/` puis reconstruire l'image ; le script n'est pas modifié.

Exemple : job commandes

```
-- commandes_abandonnees_france.sql (détection)
SELECT c.id, c.statut, cl.pays
FROM commande c LEFT JOIN client cl ON cl.id = c.client_id
WHERE c.statut = 'ABONDONNE' AND cl.pays = 'France';

-- maj_statut_ok.sql (mise à jour)
UPDATE commande SET statut = 'OK' WHERE id IN ({{IDS}});
```

4. Détails techniques

Script select_update.sh

Élément	Comportement
`set -euo pipefail`	arrêt sur erreur, variable non définie ou échec dans un pipe
`run_sql / run_sql_file`	client mysql en mode batch (`-N -B`), requête en -e ou via stdin
`render_sql`	`sed` remplace {{IDS}} par la liste d'id
`list_sql`	classe les fichiers d'un dossier par préfixe (maj, detail, select)
`run_job`	valide la convention puis enchaîne select, liste, détail, update, détail
`log`	écrit à l'écran et dans le fichier du jour (`tee -a`)

Configuration (variables d'environnement)

Variable	Défaut	Rôle
DB_HOST	localhost	hôte MySQL
DB_USER	root	utilisateur
DB_PASSWORD	root	mot de passe
DB_NAME	ecommerce	base
LOG_DIR	/logs	dossier de logs (dans le conteneur)
SQL_DIR	/usr/local/share/sql	dossier des jobs
RETENTION_JOURS	7	durée de conservation des logs

Le fichier `.env` (modele : `.env.example`, non versionné) est lu par docker compose : DB_PASSWORD alimente le mot de passe root du serveur, `LOG_DIR` désigne le dossier hôte monté sur `/logs`. Si DB_USER n'est pas root, `init_user.sh` crée l'utilisateur à l'initialisation du volume.

Traçabilité

- **Log quotidien** `sql_updates_AAAA-MM-JJ.log` : chaque lancement ajoute un bloc `=== Lancement a HH:MM ===` avec fichiers SQL utilisés, lignes détectées, liste d'id, requête exécutée, détail avant/après.
- **Table historique** : un trigger AFTER UPDATE par table (client, produit, commande, livraison) y enregistre l'état de la ligne avant et après chaque modification, quel que soit l'outil qui la fait.

```
=== Lancement à 19:50 (2026-10-14) ===
[commandes] Fichiers SQL : select=... | update=... | detail=...
--- Lignes détectées ---
4■ABONDONNE■France
--- Liste des id ---
4,9,15
--- Requête exécutée (commandes/maj_statut_ok.sql) ---
UPDATE commande SET statut = 'OK' WHERE id IN (4,9,15);
```

5. Déploiement et exploitation

```
cp .env.example .env              # adapter utilisateur, mot de passe, base, logs
docker compose up -d --build      # construire et démarrer
docker compose ps                 # attendre l'état healthy
docker compose exec mysql select_update.sh              # tous les jobs
docker compose exec mysql select_update.sh livraisons   # un seul job
crontab crontab                   # planification (heures pleines 6h-22h)
```

Planification

```
0 6-22 * * * cd /chemin/du/projet && docker compose exec -T mysql select_update.sh >/dev/null 2>&1
```

Le cron tourne sur l'hôte et exige que le conteneur soit démarré. Les valeurs du .env ne sont lues qu'à la première initialisation du volume de données : pour les changer ensuite, `docker compose down -v` (supprime les données) puis redémarrer.

6. Limites et pistes d'amélioration

Point d'attention	Détail
Pas de transaction globale	Chaque UPDATE est indépendant ; aucun rollback entre les étapes d'un job.
Mot de passe en ligne de commande	`-p$DB_PASSWORD` est visible dans la liste des processus du conteneur. Alternative : fichier d'options ou `MYSQL_PWD`.
Erreurs masquées	`2>/dev/null` cache les messages d'erreur MySQL ; un échec SQL est difficile à diagnostiquer dans le log.
Injection du marqueur	{{IDS}} est remplacé par des valeurs issues de la base (id numériques) ; ne pas l'alimenter avec des données libres.
Exécutions concurrentes	Pas de verrou : deux lancements simultanés peuvent traiter les mêmes lignes. Alternative : `flock`.
Valeurs par défaut	root/root conviennent pour un test local uniquement.

Ces points n'empêchent pas l'usage actuel (données de démonstration) ; ils sont à traiter avant un usage en production.